

Problem Statement: Enterprise Employee Resource Management System (ERMS)

Overview

You are required to design and develop a **single-page React application** for a large enterprise to manage **employees, departments, projects, and access control**.

The application must be built **only using React Functional Components and Hooks**.

Class components are strictly not allowed.

The application should follow **production-level folder structure**, use **React Router, Bootstrap for UI**, and implement **proper data flow between components**.

Core Objectives

- Build a **multi-page React SPA** using **React Router**
- Use **Bootstrap** for layout, responsiveness, header, footer, forms, and tables
- Use **React Hooks only** (useState, useEffect, useContext, useParams, useNavigate)
- Pass data:
 - Parent → Child (props)
 - Child → Parent (callback props)
 - Across pages (route params & shared context)
- Implement **role-based routing**
- Maintain **global state** using Context API
- Follow clean component separation

Application Features & Modules

1. Layout Components (Persistent Across Pages)

Header Component

- Displays:
 - Application name
 - Logged-in user name & role
 - Navigation menu:
 - Dashboard
 - Employees
 - Departments
 - Projects
 - Profile
 - Logout

- Navigation links must use **React Router**
- Header must receive **user data from Context**

Footer Component

- Displays:
 - Company copyright
 - Version info
- Visible on **all pages**

2. Authentication & User Context

◆ Login Page

- User enters:
 - Username
 - Role (Admin / Manager / Employee)
- On successful login:
 - Store user info in **Context**
 - Redirect to Dashboard
- No backend required (mock data only)

◆ UserContext

- Stores:
 - Logged-in user details
 - Role
 - Login status
- Used by:
 - Header
 - Protected routes
 - Dashboard

3. Routing Structure

Route	Component
/login	LoginPage
/dashboard	Dashboard

Route	Component
/employees	EmployeeList
/employees/:id	EmployeeDetails
/departments	DepartmentList
/projects	ProjectList
/profile	UserProfile
*	NotFound

All routes except /login must be **protected**.

4. Dashboard Page

Dashboard Component

- Displays summary cards:
 - Total Employees
 - Total Departments
 - Active Projects
- Data must be:
 - Passed from a **parent App-level state**
 - Rendered as reusable **StatCard components**
- Uses useEffect to simulate API data loading

5. Employee Management Module

EmployeeList Component

- Displays list of employees in a Bootstrap table
- Each employee row has:
 - View button
 - Delete button (Admin only)
- Clicking **View** navigates to EmployeeDetails page
- Employee data passed via:
 - Route param (id)
 - Lookup from shared state

EmployeeDetails Component

- Displays:

- Employee Name
 - Department
 - Assigned Projects
- Must:
 - Read employee ID using useParams
 - Fetch employee from shared data
- Contains a child component:
 - **ProjectAssignment**
 - Allows assigning a project
 - Sends updated project info back to parent

6. Department Module

DepartmentList Component

- Displays departments
- On click:
 - Show employees belonging to that department
- Department → Employee relationship handled using **props and filtering logic**

7. Project Module

ProjectList Component

- Displays list of projects
- Each project shows:
 - Project name
 - Assigned employees count
- Clicking a project:
 - Displays project details in a child component

8 Profile Page

UserProfile Component

- Displays logged-in user info
- Data must come from **Context**
- Allows updating display name (local state)

9. Role-Based Access Control

Role	Permissions
------	-------------

Admin	Full access (delete employees, all pages)
-------	---

Manager	View employees, departments, projects
---------	---------------------------------------

Employee	View profile and dashboard only
----------	---------------------------------

Unauthorized access must redirect or show **Access Denied**.

Technical Constraints

- ✓ Only React Hooks
- ✓ No Redux
- ✓ No class components
- ✓ Bootstrap only for UI
- ✓ No backend (mock data / static JSON)

Suggested Folder Structure

src/

```
├── components/
│   ├── Header.jsx
│   ├── Footer.jsx
│   ├── ProtectedRoute.jsx
│   └── StatCard.jsx
├── pages/
│   ├── Login.jsx
│   ├── Dashboard.jsx
│   ├── Employees/
│   │   ├── EmployeeList.jsx
│   │   └── EmployeeDetails.jsx
│   ├── Departments.jsx
│   └── Projects.jsx
```

```
| └─ Profile.jsx
|
| └─ context/
|
| └─ UserContext.jsx
|
| └─ data/
|
| └─ mockData.js
|
| └─ App.jsx
└─ main.jsx
```

Evaluation Criteria

- Correct hook usage
- Routing & navigation
- Clean data flow
- Bootstrap responsiveness
- Role-based access control
- Code quality & structure